



Cambridge IGCSE™

COMPUTER SCIENCE

0478/21

Paper 2 Problem Solving and Programming

May/June 2022

MARK SCHEME

Maximum Mark: 50

Published

This mark scheme is published as an aid to teachers and candidates, to indicate the requirements of the examination. It shows the basis on which Examiners were instructed to award marks. It does not indicate the details of the discussions that took place at an Examiners' meeting before marking began, which would have considered the acceptability of alternative answers.

Mark schemes should be read in conjunction with the question paper and the Principal Examiner Report for Teachers.

Cambridge International will not enter into discussions about these mark schemes.

Cambridge International is publishing the mark schemes for the May/June 2022 series for most Cambridge IGCSE, Cambridge International A and AS Level and Cambridge Pre-U components, and some Cambridge O Level components.

This document consists of **13** printed pages.

PUBLISHED**Generic Marking Principles**

These general marking principles must be applied by all examiners when marking candidate answers. They should be applied alongside the specific content of the mark scheme or generic level descriptors for a question. Each question paper and mark scheme will also comply with these marking principles.

GENERIC MARKING PRINCIPLE 1:

Marks must be awarded in line with:

- the specific content of the mark scheme or the generic level descriptors for the question
- the specific skills defined in the mark scheme or in the generic level descriptors for the question
- the standard of response required by a candidate as exemplified by the standardisation scripts.

GENERIC MARKING PRINCIPLE 2:

Marks awarded are always **whole marks** (not half marks, or other fractions).

GENERIC MARKING PRINCIPLE 3:

Marks must be awarded **positively**:

- marks are awarded for correct/valid answers, as defined in the mark scheme. However, credit is given for valid answers which go beyond the scope of the syllabus and mark scheme, referring to your Team Leader as appropriate
- marks are awarded when candidates clearly demonstrate what they know and can do
- marks are not deducted for errors
- marks are not deducted for omissions
- answers should only be judged on the quality of spelling, punctuation and grammar when these features are specifically assessed by the question as indicated by the mark scheme. The meaning, however, should be unambiguous.

GENERIC MARKING PRINCIPLE 4:

Rules must be applied consistently, e.g. in situations where candidates have not followed instructions or in the application of generic level descriptors.

GENERIC MARKING PRINCIPLE 5:

Marks should be awarded using the full range of marks defined in the mark scheme for the question (however; the use of the full mark range may be limited according to the quality of the candidate responses seen).

GENERIC MARKING PRINCIPLE 6:

Marks awarded are based solely on the requirements as defined in the mark scheme. Marks should not be awarded with grade thresholds or grade descriptors in mind.

Please note the following further points:

The words in **bold** in the mark scheme are important text that needs to be present, or some notion of it needs to be present. It does not have to be the exact word, but something close to the meaning.

If a word is underlined, this **exact** word must be present.

A single forward slash means this is an alternative word. A double forward slash means that this is an alternative mark point.

Ellipsis (...) on the end of one-mark point and the start of the next means that the candidate **cannot** get the second mark point without being awarded the first one. If a mark point has an ellipsis at the beginning, but there is no ellipsis on the mark point before it, then this is just a follow-on sentence and **can** be awarded **without** the previous mark point.

Question	Answer	Marks
Section A		
1(a)	One mark per mark point, max five Data Structure(s), max two MP1 arrays MP2 variable(s) / constant(s) Further description, max three MP3 name(s) one or more MP4 sample data for appropriate arrays or variables MP5 use(s) one or more Additional data structure description using the same data structure type, max one MP6 two or more full descriptions of the data structure including name, sample data and use For example: An array (1) named FirstName (1) to store the first names of the members (1) such as James (1). A variable (1) could also be used to enter whether or not they wish to volunteer. (5 marks) A variable (1) named FirstName (1) to input the first names of the members (1) such as James (1). A variable could also be used to enter whether or not they wish to volunteer, with sample data of 'yes' (1). (5 marks) Task 1 – becoming a member of Friends of Seaview Pier Set up a system to enable people to become members of Friends of Seaview Pier and for each new member enter: • their first name and last name • whether or not they wish to work as a volunteer – if they choose to volunteer, identify the area from: ○ the pier entrance gate ○ the gift shop ○ painting and decorating • the date of joining • whether or not they have paid the \$75 fee. All of this information needs to be stored using suitable data structures.	5

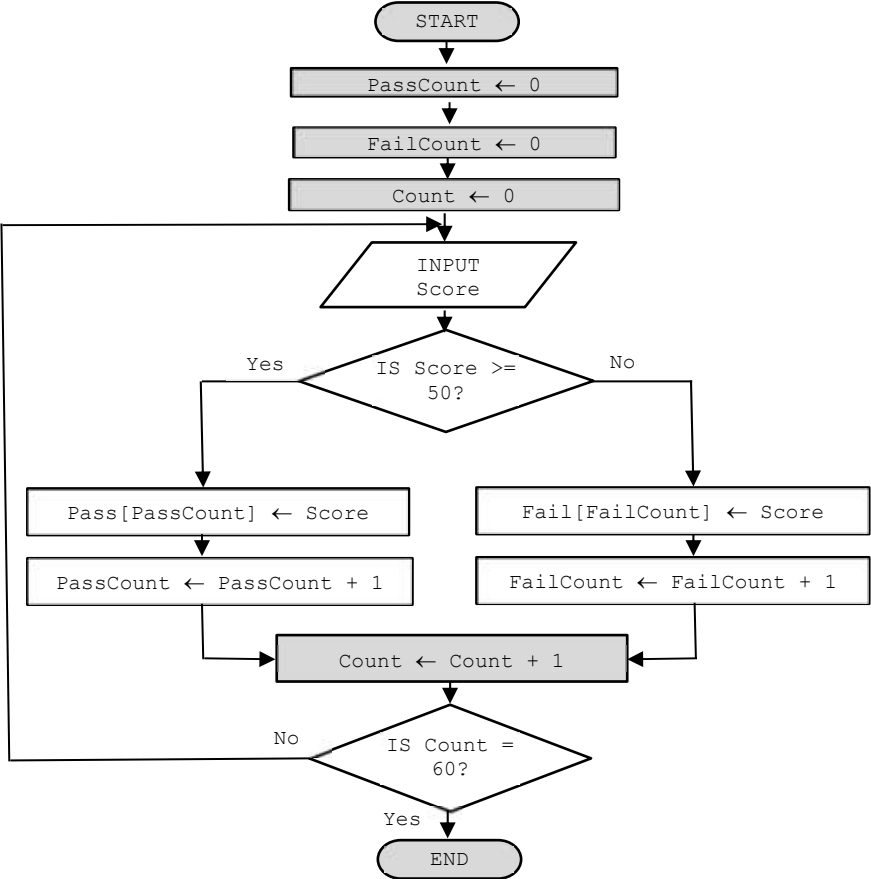
Question	Answer	Marks
1(b)	Explanation of how each was done. Code is allowed, but must be fully explained. One mark per mark point, max three MP1 add/use a (new) variable/array to total the membership fee money // total number of members who have paid MP2 initialise the totalling variable to 0 at the start of the program MP3 check whether the new member has paid the \$75 fee using a conditional statement MP4 ...if they have paid, and the amount paid is being totalled, add 75 to the current running total of the membership fee total MP5 ...if they have paid, and the number of paid members is being totalled, add 1 to the total number of members who have paid MP6 ...if they have paid, and the number of paid members is being totalled, multiply total by 75, to give total paid.	3
1(c)	Any code must include a description of what it is for. One mark per mark point, max three Max two for just naming different appropriate validation checks MP1 apply a presence check // to ensure that data has been entered (to the question do you want to work as a volunteer?) MP2 apply a type check // to ensure that data has been entered of the correct data type e.g. integer if the response required is 1 or 0 MP3 checking the valid inputs would be either yes or no // apply a check to ensure that the data matches the expected input MP4 the valid inputs would be to choose in which area the new member wishes to volunteer e.g. a list of areas MP5 if input is not valid, an error message is output (and a new input is requested) MP6 if input is valid, the program continues	3

Question	Answer	Marks
1(d)	One mark per mark point, max five MP1 input for sponsor's name / name taken from previous data // input for message on plaque MP2 both inputs correct with appropriate prompts // input for message on plaque correct with appropriate prompt and name taken from previous data MP3 output of input(s) for confirmation ... MP4 ... method for sponsor to confirm that the input(s) are correct MP5 method to enable re-entry of message on plaque if errors made MP6 charge of \$200 acknowledged / charged / confirmed / displayed MP7 (name and) message stored in arrays MP8 array index incremented for next sponsor Task 3 – sponsoring a wooden plank Add an additional option to the program in Task 1 to enable the pier's wooden planks to be sponsored. Separate data structures should be used to store the names of the individuals and the short messages they would like to have written on their brass plaque. An output would display everything that was input for the sponsor to confirm. If errors are found, the program should allow data to be re-entered. Once complete, the data is stored and the sponsor is charged \$200. Example answer <pre> ArrayIndex ← 0 //initial array's index Check ← "N" WHILE Check <> "Y" OUTPUT "Enter your name" INPUT Name OUTPUT "Enter the message you would like on your brass plaque" INPUT Message OUTPUT "The data you have entered is, name: ", Name, " and your message for the plaque is: ", Message OUTPUT "Is this correct (Y or N)" INPUT Check IF Check <> "Y" THEN OUTPUT "The data entered is incorrect, please re-enter" ENDIF ENDWHILE </pre>	5

Question	Answer	Marks
1(d)	<pre> PlankName[ArrayIndex] ← Name Message[ArrayIndex] ← Message ArrayIndex ← ArrayIndex + 1 OUTPUT "The fee for this service is \$200" //some method of paying the fee or acknowledgement of the fee </pre>	
1(e)	Explanation of how each was done. Code is allowed, but must be fully explained. One mark per mark point, max four MP1 a menu is provided/options are displayed so that the user can choose which of the lists they wish to see MP2 the user inputs a number/code as shown on the menu corresponding to their menu choice MP3 attempt to validate input MP4 ... if it does not match an option, give an error message and ask for re-input MP5 ... if it matches an option, a range of IF statements/conditional statements/CASE statement are/is used to compare the input with the available options MP6 ... output the chosen list e.g. using a loop to output the contents of the appropriate first and second name arrays corresponding to the user input MP7 identification of empty list and appropriate action Task 2 – using the membership data Extend the program in Task 1 so that a list of the first and last names of members can be output in any of the following categories: • Members who have chosen to work as volunteers. • Volunteers who would like to work at the pier entrance gate. • Volunteers who would like to work in the gift shop. • Volunteers who would like to help with painting and decorating tasks. • Members whose membership has expired (they have not re-joined this year). • Members who have not yet paid their \$75 fee.	4

Question	Answer	Marks																																									
Section B																																											
2	Four marks for five correct rows Three marks for four correct rows Two marks for three correct rows One marks for two correct rows <table><tr><th rowspan="2">Description</th><th colspan="5">Data type</th></tr><tr><th>Boolean</th><th>Char</th><th>Integer</th><th>Real</th><th>String</th></tr><tr><td>a single character from the keyboard</td><td></td><td>✓</td><td></td><td></td><td></td></tr><tr><td>multiple characters from the keyboard</td><td></td><td></td><td></td><td></td><td>✓</td></tr><tr><td>only one of two possible values</td><td>✓</td><td></td><td></td><td></td><td></td></tr><tr><td>only whole numbers</td><td></td><td></td><td>✓</td><td></td><td></td></tr><tr><td>any number</td><td></td><td></td><td></td><td>✓</td><td></td></tr></table>	Description	Data type					Boolean	Char	Integer	Real	String	a single character from the keyboard		✓				multiple characters from the keyboard					✓	only one of two possible values	✓					only whole numbers			✓			any number				✓		4
Description	Data type																																										
	Boolean	Char	Integer	Real	String																																						
a single character from the keyboard		✓																																									
multiple characters from the keyboard					✓																																						
only one of two possible values	✓																																										
only whole numbers			✓																																								
any number				✓																																							

Question	Answer	Marks
3	One mark per mark point, max four • Normal test data computerscience@cambridge.org.uk • Reason this is a valid email address (containing the @ symbol) and should be accepted • Erroneous test data computerscienceisgreat • Reason this is just a string, and should be rejected (as an email address needs a single '@')	4

Question	Answer	Marks
4(a)	One mark per mark point, max six MP1 input box MP2 correct check of Score MP3 assign Score to Pass correctly MP4 assign Score to Fail correctly MP5 increment both array counters MP6 correct check of number of scores  <pre> graph TD START([START]) --> InitPass[PassCount ← 0] InitPass --> InitFail[FailCount ← 0] InitFail --> InitCount[Count ← 0] InitCount --> Input[/INPUT
Score/] Input --> Decision1{IS Score ≥ 50?} Decision1 -- Yes --> PassAssign[Pass[PassCount] ← Score] Decision1 -- No --> FailAssign[Fail[FailCount] ← Score] PassAssign --> PassInc[PassCount ← PassCount + 1] FailAssign --> FailInc[FailCount ← FailCount + 1] PassInc --> CountInc[Count ← Count + 1] FailInc --> CountInc CountInc --> Decision2{IS Count = 60?} Decision2 -- No --> Input Decision2 -- Yes --> END([END]) </pre>	6

Question	Answer	Marks
4(b)	One mark per mark point, max four MP1 appropriate conditional loop structure MP2 correct identification of invalid input MP3 appropriate error message MP4 repeated input of score until correct <pre> WHILE Score < 0 OR Score > 100 (DO) OUTPUT "Your entry must be between 0 and 100, inclusive, please try again " INPUT Score ENDWHILE Or: REPEAT IF Score < 0 OR Score > 100 THEN OUTPUT "Your entry must be between 0 and 100, inclusive, please try again " INPUT Score ENDIF UNTIL Score >= 0 AND Score <= 100 </pre>	4

Question	Answer	Marks																																																																														
5(a)	One mark per mark point, max five MP1 correct Counter and Limit columns MP2 correct Value column MP3 correct First column MP4 correct Last column MP5 correct OUTPUT <table><tr><th>Counter</th><th>Value</th><th>First</th><th>Last</th><th>Limit</th><th>OUTPUT</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>8</td><td></td></tr><tr><td>1</td><td>66</td><td></td><td></td><td></td><td></td></tr><tr><td>2</td><td>606</td><td>6</td><td>6</td><td></td><td>606</td></tr><tr><td>3</td><td>6226</td><td></td><td></td><td></td><td></td></tr><tr><td>4</td><td>8448</td><td></td><td></td><td></td><td></td></tr><tr><td>5</td><td>642</td><td>6</td><td>2</td><td></td><td></td></tr><tr><td>6</td><td>747</td><td>7</td><td>7</td><td></td><td>747</td></tr><tr><td>7</td><td>77</td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td>121</td><td>1</td><td>1</td><td></td><td>121</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Counter	Value	First	Last	Limit	OUTPUT			0	0	8		1	66					2	606	6	6		606	3	6226					4	8448					5	642	6	2			6	747	7	7		747	7	77					8	121	1	1		121																			5
Counter	Value	First	Last	Limit	OUTPUT																																																																											
		0	0	8																																																																												
1	66																																																																															
2	606	6	6		606																																																																											
3	6226																																																																															
4	8448																																																																															
5	642	6	2																																																																													
6	747	7	7		747																																																																											
7	77																																																																															
8	121	1	1		121																																																																											

Question	Answer	Marks
5(b)	One mark per mark point, max two - checks for / outputs 3-digit numbers ... where the first and last digit are the same	2

Question	Answer	Marks																																				
6(a)	8	1																																				
6(b)	The primary key field must be unique/different for each record in the table	1																																				
6(c)	One mark per mark point, max three • correct fields and table named correctly • correct sort and show box rows • correct search criteria <table><tr><td>Field:</td><td>GameID</td><td>GameName</td><td>GamePrice</td><td>NumberStock</td><td>OnOrder</td></tr><tr><td>Table:</td><td>GAMES</td><td>GAMES</td><td>GAMES</td><td>GAMES</td><td>GAMES</td></tr><tr><td>Sort:</td><td></td><td>Ascending</td><td></td><td></td><td></td></tr><tr><td>Show:</td><td>☒</td><td>☒</td><td>☒</td><td>☐</td><td>☐</td></tr><tr><td>Criteria:</td><td></td><td></td><td></td><td>=0</td><td>="Y"</td></tr><tr><td>or:</td><td></td><td></td><td></td><td></td><td></td></tr></table>	Field:	GameID	GameName	GamePrice	NumberStock	OnOrder	Table:	GAMES	GAMES	GAMES	GAMES	GAMES	Sort:		Ascending				Show:	☒	☒	☒	☐	☐	Criteria:				=0	="Y"	or:						3
Field:	GameID	GameName	GamePrice	NumberStock	OnOrder																																	
Table:	GAMES	GAMES	GAMES	GAMES	GAMES																																	
Sort:		Ascending																																				
Show:	☒	☒	☒	☐	☐																																	
Criteria:				=0	="Y"																																	
or:																																						